● COMPLETE · 10/10

API Gateway

api-gateway

Edge Routing · JWT · Load Balancing

STATUS	TESTS	BUILD	GENERATED
Frozen	10/10 Passing	SUCCESS	March 07, 2026

api-gateway — COMPLETION RECORD

STATUS: FROZEN — DO NOT MODIFY
Tests passed: 10/10 | Build: SUCCESS | Date: 2026-03-07

Service Overview

Spring Cloud Gateway edge layer for the EduTech AI Platform. Routes all inbound traffic to downstream services, enforces JWT RS256 authentication, adds request IDs, and provides rate limiting via Redis token bucket (configured via YAML). Aggregates OpenAPI docs from all downstream services via springdoc.

Test Results

```
Tests run: 10, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS — Total time: 6.554 s
```

TEST CLASS	TESTS	RESULT
ArchitectureRulesTest	5	PASS
JwtTokenValidatorTest	5	PASS

Architecture

Spring Cloud Gateway (reactive, WebFlux-based). No hexagonal layers — configuration-driven routing.

```
com.edutech.gateway/
  GatewayApplication.java          @SpringBootApplication @ConfigurationPropertiesScan
  config/
    JwtProperties.java             record, @ConfigurationProperties("jwt")
    SecurityConfig.java            @EnableWebFluxSecurity, ServerHttpSecurity
  security/
    JwtTokenValidator.java         RSA public key loader + JJWT 0.12.x validate()
    JwtAuthenticationFilter.java   GlobalFilter (order -1) — JWT enforcement
  filter/
    RequestIdFilter.java           GlobalFilter (order -2) — X-Request-ID injection
```

Routes (configured in application.yml)

ROUTE ID	URI ENV VAR	PATH PREDICATE
auth-svc	`${AUTH_SVC_URI}`	`/api/v1/auth/**`
parent-svc	`${PARENT_SVC_URI}`	`/api/v1/parents/**`
center-svc	`${CENTER_SVC_URI}`	`/api/v1/centers/**`
assess-svc	`${ASSESS_SVC_URI}`	`/api/v1/assessments/**`
psych-svc	`${PSYCH_SVC_URI}`	`/api/v1/psych/**`
ai-gateway-svc	`${AI_GATEWAY_SVC_URI}`	`/api/v1/ai/**`

Public paths (no JWT required): `/api/v1/auth/**`, `/actuator/**`, `/swagger-ui/**`, `/v3/api-docs/**`, `/webjars/**`

Key Design Decisions

- **Fully reactive**: Spring Cloud Gateway built on WebFlux. No spring-boot-starter-web.
- **JWT enforcement via GlobalFilter** (not Spring Security `ReactiveAuthenticationManager`): `JwtAuthenticationFilter` at order `-1` validates Bearer tokens for non-public paths. Spring Security's `SecurityConfig` permits all exchanges at the security layer — JWT enforcement is the filter's responsibility.
- **Request ID**: `RequestIdFilter` at order `-2` injects `X-Request-ID` UUID if not already present.
- **Downstream headers**: On valid JWT, filter forwards `X-User-Id`, `X-User-Role`, `X-User-Center-Id` to downstream services.
- **Rate limiting**: YAML-configured `RequestRateLimiter` default filter using Spring Cloud Gateway's built-in Redis token bucket (`redis-rate-limiter.*` env vars).

- **JWT RS256**: JJWT 0.12.x — loads RSA public key from PEM file at startup, `Jwts.parser().verifyWith(publicKey).requireIssuer(issuer).build().parseSignedClaims(token).getPayload()`.
- **OpenAPI aggregation**: `springdoc-openapi-starter-webflux-ui` with per-service URLs configured in YAML.
- **No Lombok**, no hardcoded values, all config via `${ENV_VAR}`.

Fixes Applied

1. **ArchUnit** `jakarta.persistence.Entity.class`: JPA is not on the api-gateway classpath. Changed `beAnnotatedWith(jakarta.persistence.Entity.class)` to `beAnnotatedWith("jakarta.persistence.Entity")` — ArchUnit's string-based annotation check resolves from bytecode metadata without needing the class on the classpath.

ArchUnit Rules (5)

1. `@ConfigurationProperties` classes reside in `config` package
2. `GlobalFilter` implementations reside in `security` or `filter` package
3. No JPA `@Entity` annotations anywhere in the gateway
4. No `@Service` annotation — only `@Component` and `@Configuration`
5. `filter` package classes do not directly access `security` package internals

---

This document is permanently frozen. Any future changes to api-gateway require a new versioned record.